Parallelism, R, and OpenMP

wrathematics (http://wrathematics.info/)

September 30, 2019

- 1 About This Document
  - 1.1 tl;dr
  - 1.2 Long Version
- 2 An Introduction to Parallelism
  - 2.1 Parallelism and Performance
- 3 Examples
  - 3.1 Fizzbuzz
  - 3.2 Fizzbuzz OpenMP
  - 3.3 Mean of a Vector
  - 3.4 Other Examples
- 4 OpenMP Pros and Cons
- 5 Miscellany
  - 5.1 R and Thread Safety
  - 5.2 Environment Variables in R
  - 5.3 Compilers
  - 5.4 OpenMP in an R Package
  - 5.5 The SIMD Keyword
  - 5.6 OpenMP Alternatives
- 6 What next?
- 7 Exercises
- 8 Thanks

WARNING: This document was not written while sober. Apparently I think I'm hilarious when I'm drunk, so don't take the tone of this document too seriously. This is not an academic document, and it makes extensive use of adult language.

1 About This Document

1.1 tl;dr

- **Version:** 2.0
- **Text License:** CC BY-SA (http://creativecommons.org/licenses/by/4.0/).
- **Code License:** public domain
- **Naughty Language:** yes

1.2 Long Version

This document is licensed under a creative commons license (http://creativecommons.org/licenses/by/4.0/).

(http://creativecommons.org/licenses/by/4.0/)

In addition, all source code presented in this document, appearing in boxes like this:

```
source_code()
```

is released into the public domain. Or if you live in some kind of fake country that doesn't recognize the public domain, you may treat it as MIT licensed. I really don't give a fuck is what I'm trying to say.

This article contains the use of adult language. I assume that only an adult would be interested in this topic, but if you're a giant weepy baby, feel free to read someone else's inferior explanation of how this stuff works.

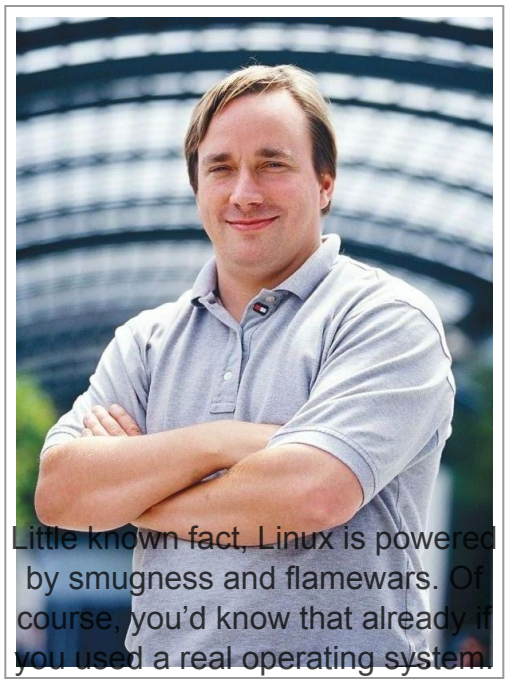
2 An Introduction to Parallelism

People talk a lot about *parallelism* these days. The basic idea is really simple. Parallelism is all about independence, literally the ability to do multiple things at the same time in a deterministic way. Mostly, parallelism isn't the hard part. Really the only reason anyone thinks this stuff is hard is because software engineering is an inherently dysfunctional discipline practiced exclusively by sociopathic masochists.

Programmers like to act like parallelism is this super complicated thing that the hoi polloi are too impossibly dumb to ever grasp. In actual fact, parallelism is really easy most of the time, especially for scientific workflows. Making good use of 15 cores might be a challenge for an Android app that interprets the stomach rumbling sounds of those near you as "hunger" or "you know damn well that guy just ate" (dear venture capitalists, call me 😉). But scientific workflows tend to be very (eheheheh) regular, and predictable.

There's a great quote about this by trollmaster Linus Torvalds that assholes often like to try to take wildly out of context:

> The whole “let’s parallelize” thing is a huge waste of everybody’s time. There’s this huge body of “knowledge” that parallel is somehow more efficient, and that whole huge body is pure and utter garbage. Big caches are efficient. Parallel stupid small cores without caches are horrible unless you have a very specific load that is hugely regular (ie graphics).
>
> …
>
> The only place where parallelism matters is in graphics or on the server side, where we already largely have it. Pushing it anywhere else is just pointless.



For the record, I work in supercomputing professionally (or as professionally as a dipshit like me can be); and I agree with him 100%. But some people like to pretend that King Linus is saying parallelism is never worth pursuing. These people are frauds and idiots. I regularly work with 1000’s of cores (with R no less!) to solve real problems that real people really care about. In case you’re wondering what he’s talking about with caches, he’s really getting at the kinds of emerging architectures coming mostly from Intel. Linus is basically saying that Xeon Phi’s aren’t going to get shipped in the next iPhone.

For basically all of this document, we’re going to be focusing on *embarrassingly parallel* problems. Something is embarrassingly parallel if you really shouldn’t have to think very hard about how to (conceptually) parallelize it. The opposite of “embarrassingly parallel” — a phrase I think should be abandoned, by the way — is “tightly coupled”. The better way to say “embarrassingly parallel” is *loosely coupled*. Use of “embarrassingly parallel” leads to stupid shit like “unembarrassingly parallel”, which makes me want to flip over a table.

A simple example comparing the two is fitting linear models. Do you want to fit a bunch of different linear models (possibly even on completely different data?). Just throw models at your computer’s cores until your electricity usage is so outrageous that the feds kick in your door because they think you’re growing pot. But what if you want to fit just one model, on a really large dataset? Not so simple now is it, smart guy?

The first is an example of an embarrassingly parallel problem. There’s no dependence between fitting two linear models. There might be some computational advantages to thinking more carefully about how the load is distributed and making use of QR updates in the model fit…but that’s not the point; I’m saying you *could* just fire the problems off blindly and call it a day. In the latter case, you basically need a parallel matrix-matrix multiply. Of

course, there are plenty of multi-threaded BLAS out there, but think about how you would write one yourself and actually get decent performance. Now think about writing one that uses not only multiple cores, but multiple nodes. It's not so obvious.

Some people might split this into additional concepts of *task parallelism* and *data parallelism*; and even just a few years ago I might have as well. But to be honest, I don't really think there is any such thing as task parallelism, and there are people way smarter than me who've said the same thing. Sorry I even brought it up.

"I know you asked for a mound of dirt, but management read an article on hackernews about how wood chips are more scalable."

Forget computers for a second because they're miserable creations designed to make us question the value of human life. Say you have 100 workers with shovels in a field. You want to dig 100 holes? Perfect, just throw one worker at each hole and you improve your time-to-hole performance 100-fold! But what if you want to dig one really big hole? Throwing 100 people at it and telling them to dig probably won't work out for you. There's lots of inter-digger communication that has to take place to make sure that they're digging in the right place, that dirt is leaving the hole correctly, all kinds of things. And never mind that what you really needed in the first place was a mound of dirt.

For clarity, you would prefer to be in the 100 hole situation. It's just easier to manage. And like any good manager, when it's all done, you get to take credit for all the hard work. And you can even fire all the workers when you're done if you want to (this is called "the cloud"). Of course real life is usually more complicated than this. It tends to be more like, you need 20 holes you need dug, and each of vastly differing sizes, and all you have is an ice cream scoop and a drill.

Parallelism in R is very simple. This is often a huge shock to people who come to R from other languages, since everything else in R is so goddamn convoluted and stupid. But it's true! The language mandates certain guarantees about the behavior of functions, for example, that make parallelism in many cases completely trivial. And it even supports some very nice utilities for very easily writing parallel code. Unfortunately, like everything else in R, there are several official interfaces, and about a hundred unofficial ones.

In R, a good first place to start thinking about parallelism is the `parallel` package. It contains two separate interfaces, one like the one from the `multicore` package and one like the `snow` package's interface. Presumably in an effort to keep things "interesting", the `parallel` package completely subsumes the `multicore` package, but *not* the `snow` package. It also makes no attempt to unify the two interfaces. But you're not using R because it's easy; you're using it because you have to, so shhh, no tears.

We're not going to talk about the `snow` side of things because it's ugly, inefficient, and stupid, and literally the only reason to care about it at this point is that it's needed for Windows users, who at this point should be grateful we even acknowledge their existence. For the `multicore` side of things, you just turn your `lapply()` calls into `mclapply()` calls:

```
index <- 1:1e5
system.time(lapply(index, sqrt))
```

```
##    user  system elapsed
##   0.031   0.003   0.034
```

```
system.time(parallel::mclapply(index, sqrt, mc.cores=2))
```

```
##    user  system elapsed
##   0.069   0.044   0.073
```

Obviously using more cores means more performance, and I am so confident in this self-evident fact, that I won't even bother to check the results of the benchmark before completing this sentence.

So that's R, but what about OpenMP? OpenMP is a parallel programming standard for the only compiled languages that matter, C, C++, and Fortran. Basically it's a way of asking the compiler to parallelize something for you. But much like that time you were playing

dungeons and dragons and got a scroll of wish, you have to be extraordinarily careful how you phrase your request. Failure to form your request carefully will likely result in chaos and destruction.

OpenMP is a set of compiler pragmas. They look like this:

```
void do_something(double *x, int i);
int i;

#pragma omp parallel for shared(x) private(i)
for (i=0; i<n; i++)
  do_something(x, i);
```

This is basically a meta-instruction to the compiler to execute the loop in parallel. If you're used to compiled code, this probably seems pretty simple. And really, it is. In fact, I didn't even have to specify the shared/private parts, which say that threads don't need private copies of `x` but do need private copies of `i`. The compiler can usually figure that out correctly by itself.

It's worth noting that, in spite of the previous comment, OpenMP is not very smart. It basically does what you tell it, even if the compiler could easily detect that what you're suggesting is a terrible idea. Say `do_something()` isn't really a parallel operation. Well too late now, sucker, enjoy the miserable hell of debugging a non-deterministic runtime error!

2.1 Parallelism and Performance

Parallelism is kind of a hot topic right now. This is due at least in part to the fact that processors haven't really gotten any faster in the last 10 years; instead, hardware vendors have been stuffing more cores, and now hardware threads, as well as fraudulent things like hyperthreads (fyi, hardware threads != hyperthreads) into chips to maintain Moore's Law.

Up and to the right means more better. But it's in 3d, so don't stand behind it, ok?

Say you have 4 cores in your fancy new laptop. Well first of all, if it's a laptop, it probably only has 2 physical cores and a couple of hyperthreads. Anyway, chances are each one of those cores runs at the same clock rate (or slower) than the last 3 or 4 computers you've bought. One of the big reasons for this is if you continue to increase the clock rate beyond 3ghz, the chances of your computer turning into a molten box of sand increase dramatically.

But not all is lost, dear consumer; there are still many good reasons to buy a new computer. For instance, hardware vendors have found some very crafty ways to be able to do more mathematical operations with processors without actually making them “faster”. Intel in particular seems to have a real love affair with creating new forms of vector instructions nobody knows how to use.

When your hipster flash-in-the-pan language is forgotten, there will still be the eternal, undying Fortran.

As noted, parallelism in R is very cheap from a programmer’s perspective. Firing off an `mclapply()` is very simple, and you can get some decent speedup for a lot of problems. But if you have 4 cores, this strategy will never get better than 4x performance improvement over the serial version. Interestingly, sometimes you can use 4 cores to get better than 4x performance improvement in a compiled language, due to things like cache effects. But the problems where this can happen are pretty rare, and likely not the problems you have.

So parallelism can get you let’s say 3.5x improvement (optimistically) on that fancy little machine of yours. But moving your R code to a compiled language like C or C++ (or Fortran, as god intended) has an opening bid of 5x if you’re unlucky. The more “R” your code is, the higher the performance improvement ceiling. It’s not uncommon to achieve speedups on the order of 20-40x, really putting that measly 3.5x to shame. No joke, one time I got a 15,000-fold improvement over a client’s R code by moving it to a carefully optimized compiled code, completely serial.

On the other hand, maybe it would be a lot of work to move the code to a compiled language (it is). And that 3.5x might take you all of a few minutes. Or maybe you can combine the two approaches. Or maybe you realize what a terrible collection of life choices you’ve made that have brought you down this dark and lonely path to thinking about cpu caches or whatever. Look, what I’m trying to say is weigh your options.

3 Examples

3.1 Fizzbuzz

A really common first problem given to people in programming interviews is fizzbuzz. It’s basically a way of quickly weeding out people who think using a pivot table in excel makes you some kind of techno spaceman from the future, and now want to be able to legally call themselves a developer.

The problem is:

> For the numbers 1 to 100, print “fizz” if the number is divisible by 3, “buzz” if the number is divisible by 5, “fizzbuzz” if the number is divisible by both 5 and 3, and otherwise print the number.

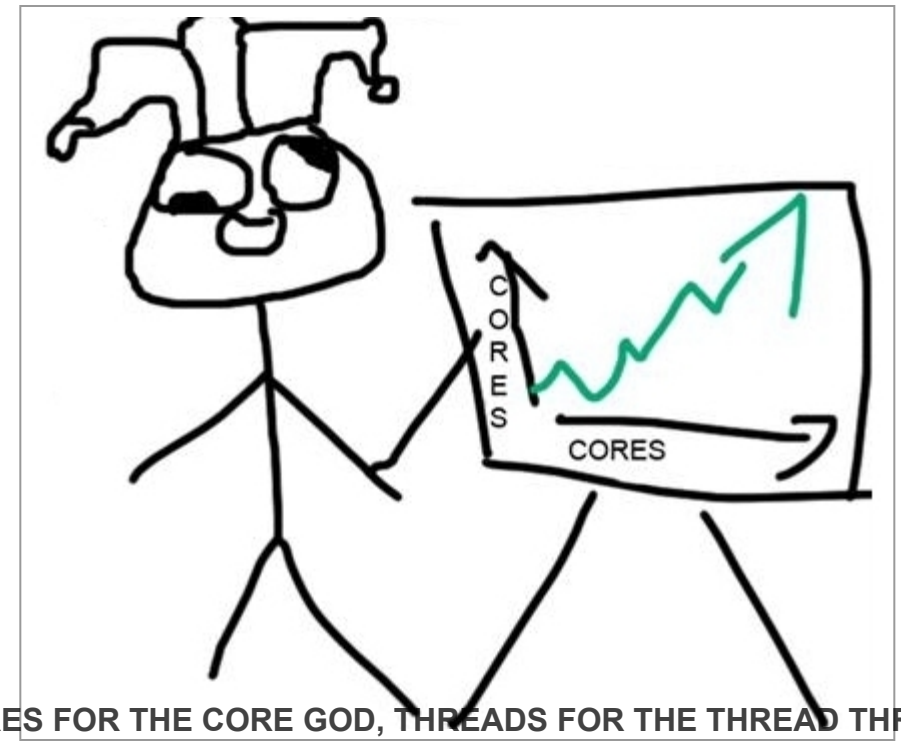
That’s honestly not the best phrasing of the problem, but it’s how I first heard it. And guess what, nerd? If your clients could figure out how to carefully phrase their problems, they wouldn’t need you and your shitty attitude.

So how might you do this in R? Lots of ways, I’m sure, but I want to impose an extra restriction: you have to use `sapply()`. Now it’s just a matter of ordering and formatting. You should come up with something reasonably similar to this:

```
fizzbuzz <- function(i)
{
  if (i%%15==0)
    print("fizzbuzz")
  else if (i%%3==0)
    print("fizz")
  else if (i%%5 == 0)
    print("buzz")
  else
    print(i)

  invisible()
}

sapply(1:100, fizzbuzz) %>% invisible()
```

```
## [1] 1
## [1] 2
## [1] "fizz"
## [1] 4
## [1] "buzz"
## [1] "fizz"
## [1] 7
## [1] 8
## [1] "fizz"
## [1] "buzz"

[[ ... results truncated ... ]]
```

Clearly this is a big data problem if I ever saw one, so what we really need is to use more cores. See figure below for details.



CORES FOR THE CORE GOD, THREADS FOR THE THREAD THRONE

If we were to use, say, `mclapply()` from the `parallel` package, we would expect to get identical outputs:

```
parallel::mclapply(1:100, fizzbuzz, mc.cores=4) %>% invisible()
```

```
## [1] 1
## [1] "buzz"
## [1] "fizz"
## [1] 13
## [1] 17
## [1] "fizz"
## [1][1] 2 "buzz"
##
## [1] 29
## [1] "fizz"

[[ ... results truncated ... ]]
```

It's perfect! Ship it!

Maybe a more common way to use `mclapply()` is to exploit the fact that it will return a list. So we could instead (conceptually) insert, as a value, the fizzbuzzed version of the integer into a vector.

```
fb <- function(i)
{
  if (i%%15==0)
    "fizzbuzz"
  else if (i%%3==0)
    "fizz"
  else if (i%%5 == 0)
    "buzz"
  else
    i
}

sapply(1:100, fb) %>% head()
```

```
## [1] "1"    "2"    "fizz" "4"    "buzz" "fizz"
```

As before, just replace `sapply()` with `mclapply()` ...then run `simplify2array()` . We can easily prove that the returns are identical:

```
n <- 100

serial <- sapply(1:n, fb)
parallel <- parallel::mclapply(1:n, fb) %>% simplify2array()

all.equal(serial, parallel)
```

```
## [1] TRUE
```

So next time you're at a job interview and they ask you to do a fizzbuzz, smugly smile and tell them that you operate at a higher level.

3.2 Fizzbuzz OpenMP

Ok so now you're a parallel expert for R. But what about OpenMP? For the sake of "you've probably dealt with it before", we'll use Rcpp. Before doing so, we have to set some compiler flags:

```
Sys.setenv("PKG_CXXFLAGS"="-fopenmp -std=c++11")
```

These tell the compiler to use OpenMP and C++11. We're using C++11 because I don't want to terrify 99% of you with the eldritch, labyrinthine and eternal horror of working with strings at a low level. Anyway, here's how you might fizzbuzz in parallel with Rcpp, returning a string vector of values:

```
#include <Rcpp.h>
#include <string>

// [[Rcpp::export]]
std::vector<std::string> fizzbuzz(int n)
{
  std::vector<std::string> ret;

  for (int i=1; i<=n; i++)
  {
    i%15 == 0 ? ret.push_back("fizzbuzz")
      : i%5 == 0 ? ret.push_back("buzz")
          : i%3 == 0 ? ret.push_back("fizz")
    : ret.push_back(std::to_string(i));
  }

  return ret;
}
```

We can now easily call the function from R:

```
fizzbuzz(100) %>% head()
```

```
## [1] "1"    "2"    "fizz" "4"    "buzz" "fizz"
```

For the remainder, we're not going to work with strings.

Fuck strings.

3.3 Mean of a Vector

Let's say we have:

```
n <- 1e8
x <- rnorm(n)
```

So in R you can use `mean()` :

```
system.time(mean(x))
```

```
##    user  system elapsed
##   0.177   0.000   0.178
```

And in serial, we can compute the mean in Rcpp like so:

```
#include <Rcpp.h>

// [[Rcpp::export]]
double mean_serial(Rcpp::NumericVector x)
{
  double sum = 0.0;
  for (int i=0; i<x.size(); i++)
    sum += x[i];

  return sum / x.size();
}
```

And in parallel using OpenMP by just adding the basic compiler pragma:

```
#include <Rcpp.h>

// [[Rcpp::export]]
double mean_parallel(Rcpp::NumericVector x)
{
  double sum = 0.0;
  #pragma omp parallel for simd shared(x) reduction(+:sum)
  for (int i=0; i<x.size(); i++)
    sum += x[i];

  return sum / x.size();
}
```

The only real trick here is making use of the `reduction` keyword, and it pretty much does what it looks like. We're telling the compiler that while each iteration of the loop should be run in parallel, in the end we want the private (by thread) copies of the variable `sum` to be added up. Now you might wonder why we didn't explicitly declare that `sum` was private, via the `private()` keyword. Well **obviously** this will create an error, because OpenMP can infer that `sum` must be private since it's part of a reduction…and OpenMP considers redundant information worthy of an error, I don't know.

The only other thing we're exploiting is the OpenMP 4 standard's `simd` keyword, which I have a lot more to say about in the later sections. Basically it uses magical parts of your processor to make things faster.

Anyway, calling them from R we see:

```
system.time(mean_serial(x))
```

```
##    user  system elapsed
##   0.295   0.001   0.297
```

```
system.time(mean_parallel(x))
```

```
##    user  system elapsed
##   0.296   0.004   0.030
```

We could add checks for `NA` or whatever like R's `mean()` does to make the comparison more fair, but I'm drunk and quickly losing interest.

3.4 Other Examples

See the Romp (https://github.com/wrathematics/Romp) package.

4 OpenMP Pros and Cons

If you're only familiar with parallelism in R, you will probably find threads very fast. There is significantly less data copying, for one. Also threads are MUCH cheaper to instantiate than sockets or forks as with the `parallel` package (though they're not completely free). Another great benefit is that if you're using R, you probably have very regular scientific workflows; this is exactly the kind of thing OpenMP was designed for, as it turns out. Finally, it's portable, and you don't have to deal with the disgusting `snow` interface.

The most obvious downside is that you have to use compiled code; otherwise this isn't really possible. Now if I were an optimist (and everything is provably terrible, so I'm not), I would say that you could use this as an excuse to start learning about, say, integrating Rcpp with your projects. But there's a reason ice cream sells more than vegetables.

The other major downside is that threading can get really *weird* if you aren't careful. By weird, I mean, you stare at it convinced with righteous certainty that it will behave in a way that you understand. Then it does something completely different and you begin to question the very nature of reality and become a solipsist monk.

OpenMP is also notorious for its difficulty to debug. This is because, as we have seen, it works via compiler pragmas. The problem is, the compiler is generating a lot of code that you really don't understand. If you're used to C++, you probably aren't worried because you're already brain-damaged from staring at million line template explosions. But in a language like C, if you really understand how your hardware works, you can generally get a rough idea of how your code will translate to assembly. This is no longer so when using OpenMP. So sometimes you just sit there scratching your head wondering if the thing even did what you just told it to do. Probably the number one thing I find myself thinking when using OpenMP is "did that even run in parallel?" This is why the OpenMP hello world is actually useful:

```
#include <omp.h>

void hellomp()
{
  // thread id and number of threads
  int tid, nthreads;

  #pragma omp parallel private(tid)
  {
    nthreads = omp_get_num_threads();
    tid = omp_get_thread_num();

    printf("Hello from thread %d of %d\n", tid, nthreads);
  }
}
```

By calling `#pragma omp parallel` in this way, we are instructing the compiler to have each thread run the section following. Otherwise this should be fairly self-explanatory. If you need to test that OpenMP is working, this is always a great one to bring out.

Finally, be careful with your core management. If you're combining `mclapply()` calls and code using OpenMP, you need to be careful with how you allocate your cores, and where. You might be ok just throwing caution to the wind, or you might need to be super careful about who gets what resources. Life is hard.

5 Miscellany

5.1 R and Thread Safety

From the Writing R Extensions manual (http://cran.r-project.org/doc/manuals/r-patched/R-exts.html#OpenMP-support):

> Calling any of the R API from threaded code is 'for experts only': they will need to read the source code to determine if it is thread-safe.

In general, `SEXP` functions (including most things you would write with Rcpp) should not be considered thread safe if you want to be cautious (and you probably do, because thread-safety problems are very hard to debug). Of note, memory allocation and the RNG are not thread safe. And it's not a bad idea to assume that every R function performs some memory allocation. What this means is, you can't call R functions in parallel. Well I mean, you *can*, but you shouldn't.

After the first publishing of this post, there was some confusion about what I mean by this. And rightly so, it was poorly phrased and overly abstract. You can use OpenMP effectively with R. Focusing on Rcpp for the moment (since that is how we have been integrating compiled code), all of the above still holds. There are examples dating back to at least 2010 effectively using OpenMP with Rcpp. There is also an example in this document, and there are yet more in the Romp package quoted by this document, and there are exercises in this document instructing you to create functions marrying Rcpp and OpenMP. It's definitely possible to do this safely.

But it's also possible to really screw up! So what can go wrong? Here's a simple example computing the column sums of the square root of a matrix (note: don't do this):

```
#include <Rcpp.h>


double rcpp_rootsum_j(Rcpp::NumericVector x)
{
  Rcpp::NumericVector ret = sqrt(x);
  return sum(ret);
}

// [[Rcpp::export]]
Rcpp::NumericVector rcpp_rootsum(Rcpp::NumericMatrix x)
{
  const int nr = x.nrow();
  const int nc = x.ncol();
  Rcpp::NumericVector ret(nc);

  #pragma omp parallel for shared(x, ret)
  for (int j=0; j<nc; j++)
    ret[j] = rcpp_rootsum_j(x.column(j));

  return ret;
}
```

Let's say we save this as `rootsum.cpp` . To call it from R, we might do something like:

```
library(Rcpp)

Sys.setenv("PKG_CXXFLAGS"="-fopenmp")
sourceCpp(file="rootsum.cpp")

m <- 100
n <- 5000
x <- matrix(runif(m*n), m, n)
all.equal(rcpp_rootsum(x), colSums(sqrt(x)))
```

Let's try running this and see what we get:

```
Error: segfault from C stack overflow
Error in print(all.equal(rcpp_rootsum(c(0.211180105920408, 0.276234067
745616,  :
  error in evaluating the argument 'x' in selecting a method for funct
ion 'print':
Execution halted
```

Hmm, that doesn't look good…let's try again:

```
terminate called after throwing an instance of 'Rcpp::not_a_matrix'
  what():  not a matrix
Aborted
```

Maybe that's just your opinion. One more time just to be sure:

```
[1] TRUE
```

Hah, I knew it worked! Maybe one more time:

```
 *** caught segfault ***
address 0x18, cause 'memory not mapped'

 *** caught segfault ***
address 0x18, cause 'memory not mapped'

 *** caught segfault ***
address 0x7f0300000000, cause 'memory not mapped'
Execution halted
Segmentation fault
```

…ehh, it's probably fine.

So what went wrong? Just like you would in plain R, we're creating copies of data behind the scenes in that column extraction, implicitly using parts of R that are not thread safe. Once again, this is not a defect in Rcpp; it is just something inherent to R.

We can perhaps make this more clear by getting rid of the Rcpp shell and use just R's basic C interface. Doing so, you might get something like:

```
#include <R.h>
#include <Rinternals.h>


double rootsum_j(SEXP x)
{
  double ret = 0.;

  for (int i=0; i<LENGTH(x); i++)
    ret += sqrt(REAL(x)[i]);

  return ret;
}

SEXP rootsum(SEXP x)
{
  const int nr = nrows(x);
  const int nc = ncols(x);
  SEXP ret;
  PROTECT(ret = allocVector(REALSXP, nc));
  SEXP col;

  #pragma omp parallel for
  for (int j=0; j<nc; j++)
  {
    PROTECT(col = allocVector(REALSXP, nr));
    for (int i=0; i<nr; i++)
      REAL(col)[i] = REAL(x)[i + nr*j];

    REAL(ret)[j] = rootsum_j(col);
    UNPROTECT(1);
  }

  UNPROTECT(1);
  return ret;
}
```

This is not a direct “instruction-to-instruction” translation (as much as that statement can even make sense), but it has similar problems, and those problems demonstrate themselves similarly. Anyone who “knows what they’re doing” (TM) will instantly see the warning signs here. We’re not only allocating data inside of a parallel region, but we’re making an unnecessary copy to boot. However, I have both personally and professionally seen numerous people make these kinds of mistakes as they transition from programming in higher languages like R to lower languages like C and C++. This is an important discussion, but as the topic here is about OpenMP and not working with a compile language for its own sake, a lot of what I might say about this is beyond the scope of this document.

One possible correction is to just write out the inner loop explicitly. It won't bite, don't worry. Another possibility is to do things more the C/C++ way than the R way (pointers or STL stuff). It may also be possible to collapse the `rcpp_rootsum_j()` function into a single line (at the `ret[j]` part). The C++ compiler may be crafty enough to see that the copy of the column is unnecessary. But I don't really know, and I wouldn't trust it regardless. Finally, you could just set `OMP_NUM_THREADS=1` before running (or remove the `pragma omp parallel for` line), and then the program would run correctly every time, but strictly in serial!

Finally, the relatively recent addition of Rcpp Attributes to Rcpp should make it much easier to avoid things like this much of the time, since it makes it easier to write idiomatic C++.

5.2 Environment Variables in R

R has a really useful way of setting/getting environment variables. We've already seen an example of this for setting the compiler flags. Unfortunately, they don't work with `OMP_NUM_THREADS`, which is a way to control the number of threads spawned by your parallel OpenMP loops. You could set this before starting your R session if you work in terminals. Otherwise, you will have to rig up something calling `omp_set_num_threads()` from C/C++, or accept the default (max threads on your machine). You can see something like this in the Rth (https://github.com/Rth-org/Rth/blob/master/src/threads.cpp) package.

5.3 Compilers

If you're doing development work on a Mac, you are probably using Clang. I love Clang, but it just got OpenMP support like a fucking week ago. Until Apple ships the new stuff with Xcode, you will need to install gcc to make use of OpenMP or find some way to manually install the new Clang stuff. I think you can install gcc through the various software repo things like macports though.

5.4 OpenMP in an R Package

If you're already familiar with the monumental headache of using compiled code in an R package, then using OpenMP is actually pretty simple. Writing R Extensions (http://cran.r-project.org/doc/manuals/R-exts.html#OpenMP-support) has some useful information, albeit presented in its usual "terse when you need help, verbose when you don't" style.

If you need more help, I recently created the Romp (https://github.com/wrathematics/Romp) package. It's a collection of very simple OpenMP examples in C, C++ (Rcpp), F77, and F2003. This, by the way, also serves as a useful example of integrating Fortran into an R package, which is not always that simple.

5.5 The SIMD Keyword

OpenMP 4, the most recent version, provides a ton of great new features. Probably my favorite is the SIMD construct. It allows you to *demand* that a loop use SIMD vectorization in a portable way. This differs from the numerous ways we've had in the past of daintily

flirting with the compiler to suggest it maybe hopefully do what we ask. There are lots of ways to politely suggest to a compiler that it should use vectorization. Using OpenMP's SIMD pragma says "fucking vectorize this!"

You can use it in a few ways, with some sometimes subtle differences. The main versions of interest are:

- `#pragma omp parallel for simd`
- `#pragma omp for simd`
- `#pragma omp simd`

The first says that the loop should be in parallel and use vector instructions. The second says to run the for loop in serial (in the sense of the number of cores to use) but to use vectorization. The third is useful to declare a loop inside of a `parallel for` region to use vector instructions. This is a really powerful construct for matrices, where maybe you operate over the rows and columns of a matrix:

```
#pragma omp parallel for
for (int j=0; j<n; j++)
{
  #pragma omp simd
  for (int i=0; i<m; i++)
    x(i, j) = something(i, j);
}
```

This will loop over the columns of the matrix in parallel, and within each thread, use vector registers to loop over the rows and do `something()`. The addition of vectorization can improve the performance of your bottlenecks by a factor of 2 to 4, depending on your hardware. This is *in addition* to the performance improvements you would get from parallelism. There are plenty of other interesting ways to use `simd`, such as `declare simd` for inline functions, but this should definitely get you started.

Now if you tell your compiler to use OpenMP and use a pragma that isn't known to your compiler, you'll get an error. Recent-ish versions of gcc (>= 4.9) support the `simd` pragma, and Clang's initial OpenMP release supports it as well. But if you are a huge asshole with an ancient software stack, you can still program for people living in the modern era pretty simply. In C/C++, you could use the preprocessor to easily check if the `simd` pragma is supported by checking the OpenMP version:

```
#ifndef MY_OMP_H_
#define MY_OMP_H_

#ifdef _OPENMP
#include <omp.h>
#if _OPENMP >= 201307
#define OMP_VER_4
#endif
#endif

// Insert SIMD pragma if supported
#ifdef OMP_VER_4
#define SAFE_SIMD _Pragma("omp simd")
#define SAFE_FOR_SIMD _Pragma("omp for simd")
#else
#define SAFE_SIMD
#define SAFE_FOR_SIMD
#endif

#endif
```

This way, I can insert `SAFE_SIMD` or `SAFE_FOR_SIMD` into my code, and if `simd` is supported by my OpenMP version, use it, if not, don't. All error free.

For Fortran users, you can't do this sort of thing without using an "FPP", which is generally a cascade of headaches and heartbreaks. Basically because there's really no such thing as an "FPP", and people tend to just use the CPP and hope for the best. I like Fortran, but we need to be honest with ourselves; if this level of control is important to you, then you should be using C.

5.6 OpenMP Alternatives

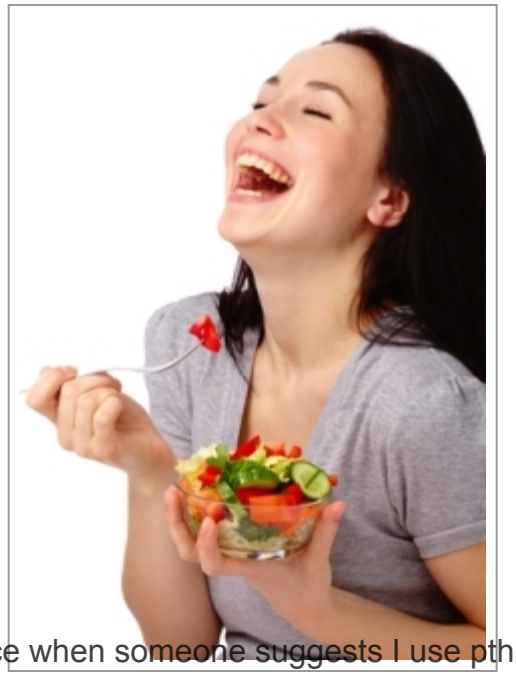
OpenACC is an alternative. Originally, OpenMP and OpenACC had vastly different goals, though more and more they look similar to me. I honestly don't know much about it because I never use it, but some people swear by it. It's probably more interesting to you if you work with accelerators a lot (I don't).

If you're interested in programming accelerator cards, like a gpu, there is an extension for OpenMP to dispatch to a gpu. I've never used it, but it seems like a bad idea to me, in the sense that I don't know how you could hope to get good performance out of it. But I've never used it, so maybe you can. In that case, I'd probably look at OpenACC, or things like the Thrust template library.

Additionally, if you're using C++, there are a *lot* of other threading frameworks out there. Most of them are shit. There's probably something in boost which, under a deeply strained legal interpretation of the word "works", works. There's also Intel Thread Building Blocks (TBB). TBB has some really cool ideas in it, but it's not portable (Intel only), and frankly by my assessment, it's pretty dead. I don't work for Intel, or have any special insight into how they do things, but I'm 99% sure they're just giving up on TBB in favor of OpenMP, which is a real and proper standard (supported deeply by Intel, no less). The various parallel

Rcpp efforts, like RcppParallel seem to be heading towards utilizing TBB (because they all use Macs, and Clang only recently got its act together on OpenMP). JJ's a super duper smart guy, but I really think he's betting on the wrong horse here.

Oh there's also pthreads.



My face when someone suggests I use pthreads

6 What next?

Norm Matloff, who I have a lot of respect and admiration for, has written a great introduction (https://matloff.wordpress.com/2015/01/16/openmp-tutorial-with-r-interface/) to OpenMP with R. I actually consider his guide a bit terse and (relatively) advanced, and it contains a lot of great orthogonal advice to the incoherent ramblings presented here. So if this didn't do it for you, give his proper guide a try. Norm has also just finished a book about parallel programming for data science (http://www.amazon.com/Parallel-Computing-Data-Science-Examples/dp/1466587016/) that I'm pretty excited about. Considering he wrote what is easily the single best book on R (http://www.amazon.com/Art-Programming-Statistical-Software-Design/dp/1593273843/), it should be great.

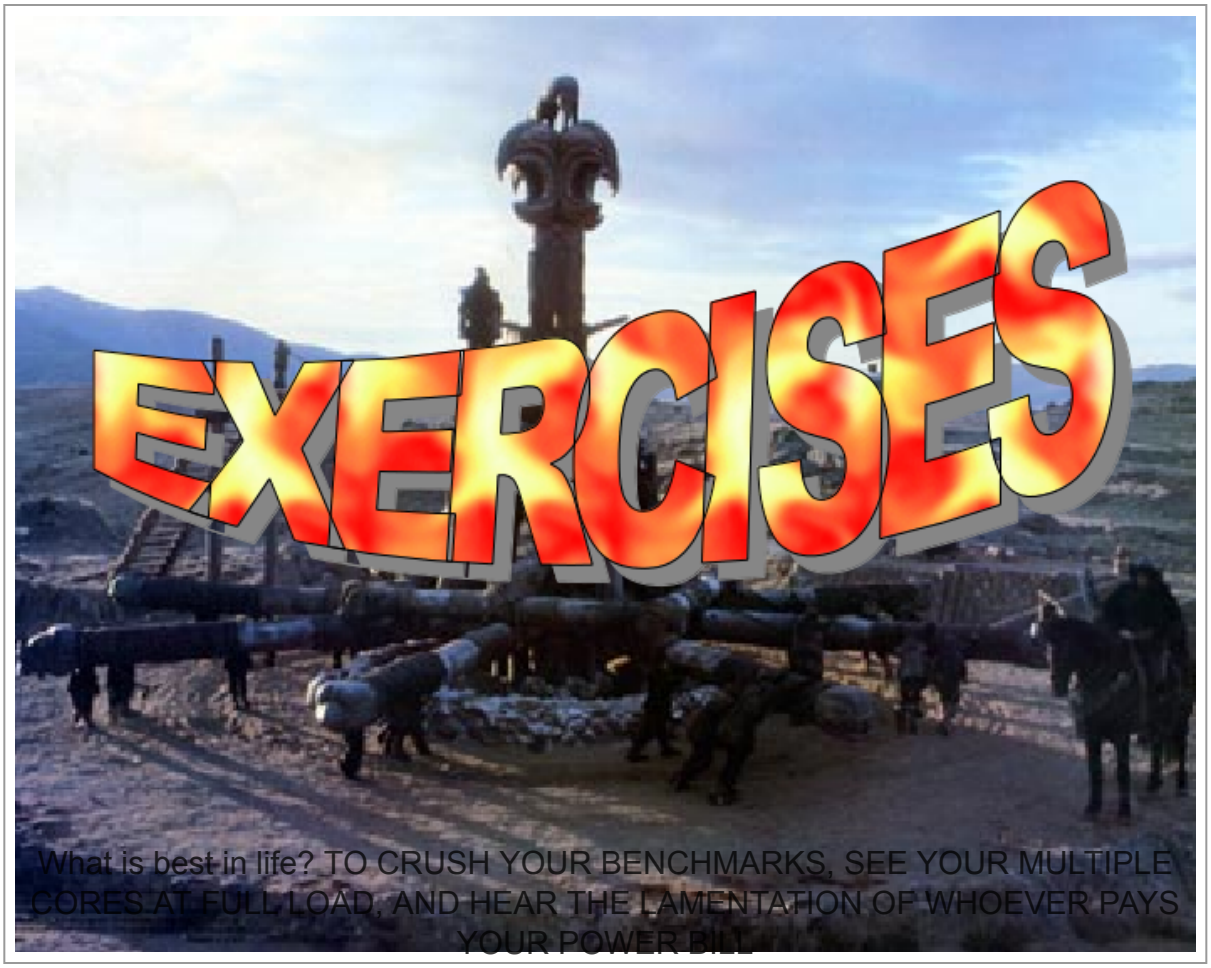
If you're looking on stuff specifically about OpenMP, the Using OpenMP (http://www.amazon.com/Using-OpenMP-Programming-Engineering-Computation/dp/0262533022/) book is pretty good. It's a pretty out of date now, written for the 2.5 standard (4.0 has been out for a bit as of the time of writing), but if you're a beginner, you should be fine starting there.

The OpenMP 4.0 Syntax Quick Reference Card (http://openmp.org/mp-documents/OpenMP-4.0-C.pdf) is useful once you kind of get the hang of OpenMP but don't use it regularly enough to remember how the order and/or presence/absence of keywords can drastically affect things.

In my opinion, books and tutorials are great to motivate you and give you a starting point, but the best way to really learn something is to dive into the deep end with a lead weight around your neck. Profile your code, find something that could be faster, and make it parallel or die trying.

We didn't talk about simulations and other things involving random number generation. Just know that it complicates things in a way I don't feel like getting into right now, and woe betide the person with such problems in the first place.

7 Exercises



For the following exercises, use your choice of C, C++, and Fortran. For C++, avoid the functional-ish STL operations and use `for` loops. When I say “write a function”, I mean “write a function in a compiled language, callable by R”, such as via Rcpp.

1. Write a function that takes a vector of integers and multiplies each element by 2.
2. Revisit the above, but using OpenMP.
3. Experiment with various problem sizes and core counts benchmarking exercises 1 and 2 above, and an R implementation (`2*x`).
4. Write a function that takes a numeric matrix and produces a numeric vector containing the sums of the columns of the matrix.
5. Revisit the above, but using OpenMP.
6. Experiment with various problem sizes and core counts benchmarking exercises 4 and 5 above, as well as the R function `colSums()` . For bonus points, handle `NA` ’s as R does.
7. Write a safe version of the `rootsum()` function in the thread safety subsection.

8 Thanks

Special thanks to:

- Karl Broman
- Joseph Stachelek
- Dirk Eddelbuettel

who each corrected and/or pointed out a number of errors, ranging from typos and grammar issues to very serious, technical flaws.

Also thanks to everyone who had kind words to say about this document. I really appreciate it.